

검진팀 실습 — 제안서 마법사에 “AI 문구 생성” 붙이기

4주차 핵심: 모델을 각자 띄우지 않습니다. **공유 게이트웨이(:11500)** 가 추론 누출(생각 토큰)·답변 길이 상한·모델명 정규화를 알아서 처리합니다. 그래서 여러분이 할 일은 단 두 가지입니다 — (1) 게이트웨이를 부르는 **연결 코드 한 덩어리 붙여넣기**, (2) 모델에게 줄 **프롬프트(맥락+규칙) 채우기**.

대상 앱: `proposal_generator_beta` (건강검진 제안서 자동 생성기) 바꿀 파일:
`core/http_handler.py` (백엔드 1곳) + `index.html` / `js` (버튼 1개)

1. 내 앱 한눈에

- **무슨 앱**: 엑셀(검진 항목·수가)을 읽어 표지·서브페이지·검진표를 합쳐 **제안서 PDF**를 만드는 4단계 마법사.
- **지금까지**: 모든 게 결정론(엑셀→PDF). AI 없음.
- **오늘 추가**: 1페이지(생성 정보)에 **[AI 문구 생성]** 버튼. 고객사명·연도·검진타입을 모델에 넘기면 **인사말/패키지 소개 문단**을 한국어로 써 줍니다.
- **건드리지 않는 것**: 표·수가·이미지 합치기(기존 PDF 생성)는 그대로. **AI는 “문장 초안”만** 담당합니다. 숫자 근거는 늘 엑셀/코드에서 나옵니다.

[1페이지 입력폼] — (고객사·연도·검진타입) —> `/api/ai-copy`

_____ |

_____ 내 서버 —urllib—> 게이트웨이

`:11500/api/chat`

_____ | _____

(`think:false`·길이상한 자동)

[textarea에 문구 채움] ← `{"copy": "..."} ←`

2. URL 한 줄 — 게이트웨이 주소만 맞추면 됩니다

이 앱은 `requests` / `httpx` 같은 외부 라이브러리가 없습니다(표준 `http.server` 만 사용). 그래서 파이썬 기본 내장 도구인 `urllib` 로 게이트웨이를 부릅니다. 아래 코드에서 바꿀 곳은 **게이트웨이 주소 한 줄**뿐입니다.

```
GATEWAY = "http://121.138.151.6:11500" # ← 수업 시작 때 강사가 알려주는 주소로 맞추세요
```

예전 방식(각자 모델 직접 띄우기)이라면 `:11434` 였습니다. 4주차부터는 공유 게이트웨이 `:11500` 한 곳만 봅니다. 끝.

3. 라우팅 한 곳 추가 — 그대로 복붙

`core/http_handler.py` 한 파일만 세 군데 손봅니다. 순서대로 복붙하세요.

3-1. 파일 맨 위 `import` 줄에 `urllib.request` 추가

기존 `import` 묶음(파일 상단, `import json` 근처)에 한 줄을 더합니다.

```
import urllib.request # ← 이 줄을 추가 (json·urllib.parse는 이미 있음)
```

3-2. 파일 맨 아래(`return Handler` 위쪽, `def _download_url` 근처)에 게이트웨이 호출 함수 붙이기

```
GATEWAY = "http://121.138.151.6:11500" # ← 수업용 게이트웨이 주소 (강사 안내대로 맞추기)
```

```
def ai_write(system: str, user: str) -> str:
    """게이트웨이(:11500)에 메시지를 보내고 생성된 문구만 돌려받는다.
    게이트웨이가 think:false·길이 상한·모델명 정규화를 알아서 처리하므로
    우리는 messages(맥락+규칙)만 잘 만들면 된다."""
    body = {
        "model": "qwen3.6:35b-a3b",
        "messages": [
            {"role": "system", "content": system},
            {"role": "user", "content": user},
        ],
        "stream": False, # 문구 생성은 한 번에 받기(논스트림)가 간단
    }
    req = urllib.request.Request(
        f"{GATEWAY}/api/chat",
        data=json.dumps(body).encode("utf-8"),
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(req, timeout=120) as r:
        data = json.loads(r.read())
        return data["message"]["content"]
```

3-3. `do_POST` 안에 라우트 한 줄 + 핸들러 메서드 추가

이 앱의 `do_POST` 는 지금 `/api/generate` , `/api/generate-final` 만 받습니다. 여기에 `/api/ai-copy` 를 끼웁니다.

먼저 `do_POST` 의 분기에 한 줄 추가:

```
def do_POST(self) -> None:
    path = urlparse(self.path).path
    if path == "/api/generate":
```

```

        self.handle_generate()
    elif path == "/api/generate-final":
        self.handle_final_generate()
    elif path == "/api/ai-copy":      # ← 이 줄 추가
        self.handle_ai_copy()        # ← 이 줄 추가
    else:
        self.send_error(404)

```

그리고 `Handler` 클래스 안(다른 `handle_...` 메서드들 옆)에 아래 메서드를 붙입니다.
 들여쓰기 8칸(`def handle_generate` 와 같은 높이) 을 꼭 맞추세요.

```

        # POST /api/ai-copy
        # 받는 것: {"customer": "...", "year": 2026, "exam_type":
        "..."}
        # 주는 것: {"copy": "<생성된 제안서 문구>"}
        def handle_ai_copy(self) -> None:
            try:
                # 1) 요청 본문(JSON) 읽기
                length = int(self.headers.get("Content-Length",
0))
                payload = json.loads(self.rfile.read(length) or b"
{}")
                customer = str(payload.get("customer",
"")).strip()
                year = str(payload.get("year", "")).strip()
                exam_type = str(payload.get("exam_type",
"")).strip()

                # 2) ▼▼▼ 여기가 오늘의 핵심 - 4번 항목에서 채웁니다 ▼▼▼
                system = SYSTEM_PROMPT
                user = build_user_prompt(customer, year,
exam_type)

                # ▲▲▲ 위 두 변수만 잘 만들면 됩니다 ▲▲▲

                copy = ai_write(system, user)
                self.send_json(200, {"copy": copy})
            except Exception as err:
                self.send_json(500, {"error": str(err)})

```

`send_json` 은 이 앱에 이미 있는 응답 헬퍼라 그대로 씁니다(추가 작업 없음).

4. 학생이 채울 곳 — 프롬프트 (오늘의 진짜 실습)

같은 모델, 같은 게이트웨이라도 **규칙을 어떻게 주느냐**로 결과가 갈립니다. 이게 “하네스”입니다. 아래 두 덩어리를 `core/http_handler.py` 맨 위(import 아래)에 추가하고, `"""` `TODO """` 부분을 직접 채우세요.

```
# -----
# TODO(학생 1): 시스템 프롬프트 = 모델이 지켜야 할 '규칙'
#   힌트로 준 뼈대를 참고하되, 빈칸을 우리 톤에 맞게 채우세요.
# -----
SYSTEM_PROMPT = """\
당신은 삼성창원병원 건강검진센터의 제안서 문구를 쓰는 작가입니다.
다음 규칙을 반드시 지키세요.
- 톤: 정중하고 신뢰감 있는 영업 제안 톤. 과장·홍보성 미사여구 금지.
- 길이: 인사말 2~3문장 + 패키지 소개 2~3문장. 전체 5문장 안팎.
- 절대 금지: 수치·검진 항목 수·가격·실적을 스스로 지어내지 말 것.
- 확실하지 않은 숫자나 항목은 그대로 쓰지 말고 [확인 필요] 라고 표시할 것.
- 출력은 완성된 한국어 문단만. 머리말('네, 작성하겠습니다' 등)·설명·목록 금지.
"""

#   ↑ TODO: 우리 센터 색깔에 맞게 한두 줄 더하거나 다듬어 보세요
#   (예: "지역 밀착", "맞춤 패키지 강조" 등). 단, '금지' 규칙은 지우지 말 것.

# -----
# TODO(학생 2): 사용자 프롬프트 = 모델에 줄 '맥락'(이번 건의 정보)
# -----
def build_user_prompt(customer: str, year: str, exam_type: str) -> str:
    return f"""\
아래 정보로 건강검진 제안서 도입 문구를 작성해 주세요.
- 고객사명: {customer}
- 검진 연도: {year}
- 검진 타입: {exam_type}
"""

#   ↑ TODO: 필요하면 줄 항목을 더 넣어 보세요(예: 대상 인원, 강조점).
#   단, 실제 고객사명·실적은 넣지 말고 더미(예: '가나물산')로 연습할 것.
```

채점 포인트(스스로 점검): 규칙(톤·길이·금지)이 분명한가? 맥락(고객사·연도·타입)이 빠짐없이 들어갔는가? “지어내지 말 것 / [확인 필요] 표시” 안전장치가 살아 있는가?

5. 프론트 — 버튼 1개 (선택, 빠른 확인용)

`index.html` 1페이지 `고객사명 / 검진기관명` 입력 아래에 버튼과 결과창을 넣습니다.

```
<div class="section-heading"><span>AI</span><div><h2>AI 문구 생성
</h2>
  <p>고객사·연도·검진타입을 바탕으로 도입 문구 초안을 만듭니다(편집 가능).</p>
</div></div>
<label>검진 타입
  <select id="ai-exam-type"><option>A타입</option><option>B타입
</option></select></label>
<button type="button" id="ai-copy-btn" class="quiet-button">AI 문구
생성</button>
<textarea id="ai-copy-out" rows="6" placeholder="생성된 문구가 여기에
표시됩니다 - 그대로 쓰거나 손보세요"></textarea>
```

`js/page1.js` (또는 `app.js`) 아무 곳에 버튼 동작을 추가합니다.

```
document.getElementById("ai-copy-btn")?.addEventListener("click",
async () => {
  const out = document.getElementById("ai-copy-out");
  out.value = "생성 중...";
  try {
    const res = await fetch("/api/ai-copy", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        customer:
document.querySelector('input[name="customer"]').value,
        year: document.getElementById("year").value,
        exam_type: document.getElementById("ai-exam-type").value,
      }),
    });
    const data = await res.json();
    out.value = data.copy || data.error || "응답이 비어 있습니다.";
  } catch (e) { out.value = "오류: " + e.message; }
});
```

버튼 없이도 됩니다. 6번의 `curl` 한 줄로 백엔드만으로 바로 확인할 수 있어요.

6. 확인 절차

1. 게이트웨이 살아있나 — 터미널에서:

```
curl http://121.138.151.6:11500/health
```

(정상 응답이 오면 통과. 안 오면 강사에게 주소 확인)

2. 앱 기동:

```
cd proposal_generator_beta
python3 server.py
```

브라우저가 <http://127.0.0.1:8765> 로 열립니다.

⚠ 이 앱은 부팅할 때 PDF 미리보기용 Node.js 렌더러를 함께 띄웁니다. 만약 기동이 `RuntimeError: Node.js 렌더링 서버 시작 실패` 로 멈추면, 그건 이 앱 원래의 렌더러 의존성(노드 번들) 문제이지 오늘 추가한 AI 기능과는 무관합니다. 처음 받은 폴더에서 노드 패키지 설치(`npm install`)가 끝나 있어야 앱이 뜹니다. 노드 렌더러가 안 떠도 AI 문구 로직 자체는 정상이며, 급하면 3번처럼 `/api/ai-copy` 만 따로 확인할 수 있습니다.

3. AI 엔드포인트 직접 호출(버튼 안 만들었어도 됨):

```
curl -X POST http://127.0.0.1:8765/api/ai-copy \
-H "Content-Type: application/json" \
-d '{"customer":"가나물산","year":"2026","exam_type":"A타입"}'
```

→ `{"copy": "..."}` 형태로 한국어 문구가 오면 성공.

4. 규칙이 지켜지는지 눈으로 확인:

- ☐ 200 응답 + `copy` 텍스트가 온다.
- ☐ 길이·톤이 시스템 프롬프트대로다(5문장 안팎, 정중함).
- ☐ 안 준 숫자를 지어내지 않는다(없는 항목 수·가격이 튀어나오면 프롬프트의 ‘금지’ 규칙 강화).
- ☐ `<think>` 같은 생각 토큰이 새지 않는다 — 게이트웨이가 막아주므로 깔끔해야 정상.
- ☐ 기존 PDF 생성(`/api/generate`)이 그대로 동작한다(회귀 없음).

7. 개선 아이디어 (여유 있으면)

- **[확인 필요] 후처리:** 생성 문구에 **[확인 필요]** 가 들어 있으면 textarea 위에 “숫자 직접 확인하세요” 경고 띄우기.
- **타입별 톤 분리:** A타입(종합)·B타입(기본)에 따라 시스템 프롬프트 한 줄을 바꿔 강조점 달리하기.
- **재생성 버튼:** 마음에 안 들면 한 번 더 호출(같은 입력, 다른 초안).
- **빈 입력 방어:** 고객사명이 비면 호출 전에 막기(불필요한 게이트웨이 호출 절약 — 공유 GPU 1개라 동시성 배려).

8. 안전수칙 (필수)

- 실제 고객사명·실적·수가·계약 내용을 모델에 넣지 마세요. 수업은 더미/비식별로만 (예: 고객사 가나물산, 연도 2026).
- 환자정보·대외비 입력 금지. AI는 “문장 초안”만, 숫자 근거는 반드시 엑셀/코드에서 나옵니다. 생성 문구의 항목 수·수가는 사람이 최종 검증.
- 공유 GPU는 1개입니다. 버튼을 연타하지 말고(요청 1번 = 모두가 줄 서는 1자리), 응답을 기다린 뒤 다시 누르세요.
- 게이트웨이 경유라 추론 토큰(생각) 누출 걱정은 없습니다 — 자동 차단됩니다. 그래서 여러분은 프롬프트에만 집중하면 됩니다.
- 생성된 문구는 편집 가능해야 합니다(사람이 검수 후 제안서에 반영). 모델 출력 그대로를 고객에게 보내지 마세요.